



北京師範大學
BEIJING NORMAL UNIVERSITY

计算方法 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 6 月 25 日

目录

1	环境说明	3
2	问题	3
3	算法	3
3.1	二分法 (Bisection Method)	3
3.2	牛顿法 (Newton-Raphson Method)	3
3.3	割线法 (Secant Method)	3
3.4	Aitken 加速牛顿法	4
4	结果	4

1 环境说明

本文全部代码为 c++ 代码，硬件为 Macbook Air M2，软件为 Xcode。在 Xcode 内部进行基于命令行的编译运行。

本实验报告使用 $\text{\LaTeX}$ 进行制作，基于 overleaf 网站进行在线写作。

2 问题

每个方程有一个实根，分别用二分法、牛顿法和割线法求近似根，精确到小数点后 8 位，然后对牛顿法采用 Aitken 加速。方程如下：

1. $\sin x = 6x + 5$

2. $\ln x + x^2 = 3$

3 算法

本实验使用四种经典迭代方法，简要介绍如下：

3.1 二分法 (Bisection Method)

该方法利用连续函数在区间上的介值定理，对有符号变化的区间反复二分，逐步逼近根。其收敛速度为线性，误差每次大致减少一半，适合用于初始区间已知包含根的情形。

3.2 牛顿法 (Newton-Raphson Method)

该方法使用泰勒展开的一阶近似，迭代公式为：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

该方法具有局部二阶收敛速度，若初始值足够接近实根，收敛非常快，但对导数和初值较敏感。

3.3 割线法 (Secant Method)

该方法近似地代替牛顿法中的导数，使用两点构成的割线，迭代公式为：

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

它不需要导数，收敛速度略慢于牛顿法，为超线性。

3.4 Aitken 加速牛顿法

为加速迭代，我们对牛顿法生成的序列使用 Aitken Δ^2 加速方法，公式为：

$$x_{\text{Aitken}} = x_n - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2x_{n+1} + x_n}$$

若迭代序列收敛，则该加速方法可以大幅提高收敛速度。但当分母太小时（例如三点接近共线），需要避免除以接近零的数。

4 结果

方程 1: $\sin x - 6x - 5 = 0$

方法	收敛结果	迭代次数
二分法	-0.9708989235	29
牛顿法	-0.9708989235	4
割线法	-0.9708989235	5
Aitken 加速牛顿法	-0.9708989235	3

表 1: 方程 1 各方法求解结果比较

输出结果图片如下：

```
Equation 1: sin(x) - 6x - 5
Bisection method:
Iter 1: x = 0.0000000000
Iter 2: x = -0.5000000000
Iter 3: x = -0.7500000000
Iter 4: x = -0.8750000000
Iter 5: x = -0.9375000000
Iter 6: x = -0.9687500000
Iter 7: x = -0.9843750000
Iter 8: x = -0.9765625000
Iter 9: x = -0.9726562500
Iter 10: x = -0.9707031250
Iter 11: x = -0.9716796875
Iter 12: x = -0.9711914062
Iter 13: x = -0.9709472656
Iter 14: x = -0.9708251953
Iter 15: x = -0.9708862305
Iter 16: x = -0.9709167480
Iter 17: x = -0.9709014893
Iter 18: x = -0.9708938599
Iter 19: x = -0.9708976746
Iter 20: x = -0.9708995819
Iter 21: x = -0.9708986282
Iter 22: x = -0.9708991051
Iter 23: x = -0.9708988667
Iter 24: x = -0.9708989859
Iter 25: x = -0.9708989263
Iter 26: x = -0.970898965
Iter 27: x = -0.9708989114
Iter 28: x = -0.9708989188
Iter 29: x = -0.9708989225
Total iterations: 29

Newton-Raphson method:
Iter 1: x = -1.0000000000
Iter 2: x = -0.9709637742
Iter 3: x = -0.9708989238
Iter 4: x = -0.9708989235
Total iterations: 4

Secant method:
Iter 1: x = -0.9692685619
Iter 2: x = -0.9709877293
Iter 3: x = -0.9708989125
Iter 4: x = -0.9708989235
Iter 5: x = -0.9708989235
Total iterations: 5

Aitken-accelerated Newton method:
Iter 1: x = -0.9717830868
Iter 2: x = -0.9708989235
Iter 3: x = -0.9708989235
Total iterations: 3
```

图 1: 方程 1 输出结果

方程 2: $\ln x + x^2 - 3 = 0$

方法	收敛结果	迭代次数
二分法	1.5921429368	29
牛顿法	1.5921429371	5
割线法	1.5921429371	6
Aitken 加速牛顿法	1.5921429371	4

表 2: 方程 2 各方法求解结果比较

输出结果图片如下:

```
Equation 2: log(x) + x^2 - 3
Bisection method:
Iter 1: x = 1.2500000000
Iter 2: x = 1.6250000000
Iter 3: x = 1.4375000000
Iter 4: x = 1.5312500000
Iter 5: x = 1.5781250000
Iter 6: x = 1.6015625000
Iter 7: x = 1.5898437500
Iter 8: x = 1.5957031250
Iter 9: x = 1.5927734375
Iter 10: x = 1.5913085938
Iter 11: x = 1.5920410156
Iter 12: x = 1.5924072266
Iter 13: x = 1.5922241211
Iter 14: x = 1.5921325684
Iter 15: x = 1.5921783447
Iter 16: x = 1.5921554565
Iter 17: x = 1.5921440125
Iter 18: x = 1.5921382904
Iter 19: x = 1.5921411514
Iter 20: x = 1.5921425819
Iter 21: x = 1.5921432972
Iter 22: x = 1.5921429396
Iter 23: x = 1.5921427608
Iter 24: x = 1.5921428502
Iter 25: x = 1.5921428949
Iter 26: x = 1.5921429172
Iter 27: x = 1.5921429284
Iter 28: x = 1.5921429340
Iter 29: x = 1.5921429368
Total iterations: 29

Newton-Raphson method:
Iter 1: x = 1.6666666667
Iter 2: x = 1.5932929205
Iter 3: x = 1.5921432155
Iter 4: x = 1.5921429371
Iter 5: x = 1.5921429371
Total iterations: 5

Secant method:
Iter 1: x = 1.5055344199
Iter 2: x = 1.5850017951
Iter 3: x = 1.5922744300
Iter 4: x = 1.5921427392
Iter 5: x = 1.5921429371
Iter 6: x = 1.5921429371
Total iterations: 6

Aitken-accelerated Newton method:
Iter 1: x = 1.6005678024
Iter 2: x = 1.5921429106
Iter 3: x = 1.5921429371
Iter 4: x = 1.5921429371
Total iterations: 4
```

图 2: 方程 2 输出结果

分析与总结

- 所有方法最终结果在小数点后 8 位均一致，满足精度要求。
- 二分法稳定但收敛慢，均需 29 次迭代。
- 牛顿法收敛极快，4-5 次即可达到精度，但需计算导数。
- 割线法略慢于牛顿法，但无需导数，适合导数难求的情况。
- Aitken 加速显著减少迭代次数，在牛顿法本已快速收敛时进一步提升效率。